Défis en Intelligence artificielle. Reinforcement Learning (1)

Xavier Siebert

MathRO, Université de Mons

http://www.ig.fpms.ac.be/~siebertx





Kasparov vs. Deep Blue (1997)

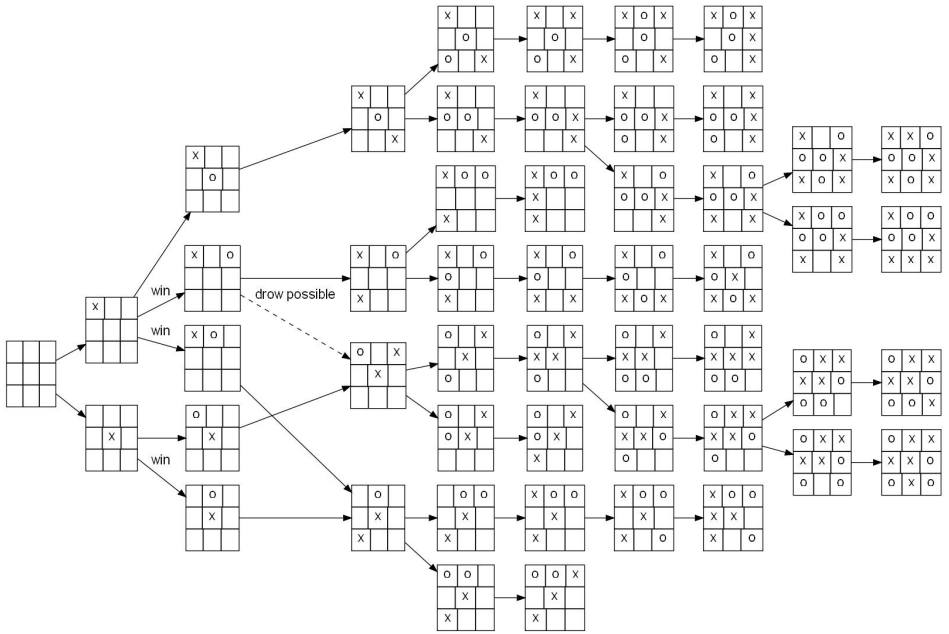


Ke Jie vs. Alpha Go Zero (2017)



Complexity of the task

- Tic-Tac-Toe : 255 168 unique games (5478 states)



Source : Wikimedia

- Chess : $\approx 10^{120}$ nodes for 80 moves
- Go : $\approx 10^{170} - 10^{360}$ nodes for 150 moves
- Number of atoms in the universe : $\approx 10^{80}$

Deep Blue (1997) - Alpha Go Zero (2017)

Computer power is important. . . but not enough !



- knowledge engineering
- evaluation function
- libraries of moves
- computational power



- deep reinforcement learning
- zero knowledge engineering
- computational power : 48 TPUs

Deep Reinforcement Learning (Deep RL)

- Deep RL = Reinforcement Learning + Deep Learning
- computational approach to learning from interaction
- can solve a wide range of complex sequential decision-making tasks that were previously out of reach for a machine.





Topics covered in this course

1. Markov Decision Processes
2. Reinforcement Learning
3. Deep Reinforcement Learning

Menu

1 Introduction

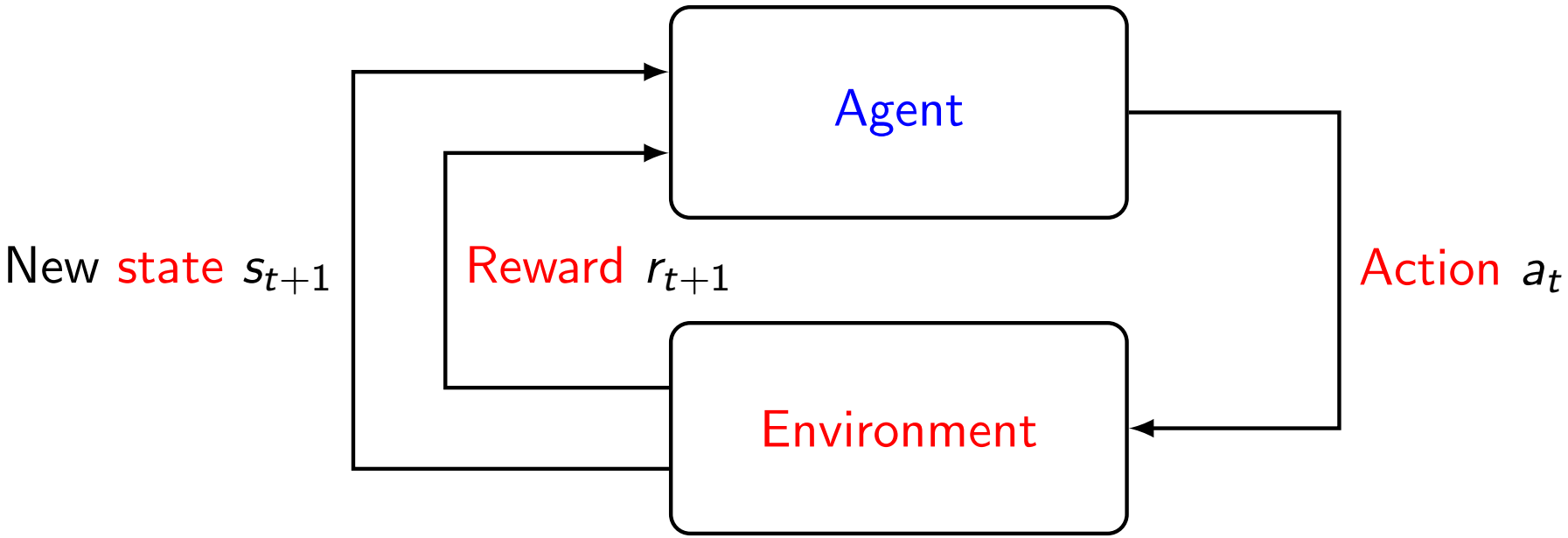
2 Markov Decision Processes

3 Bellman Equations

4 Value Iteration

5 Policy Iteration

Reinforcement Learning Scheme



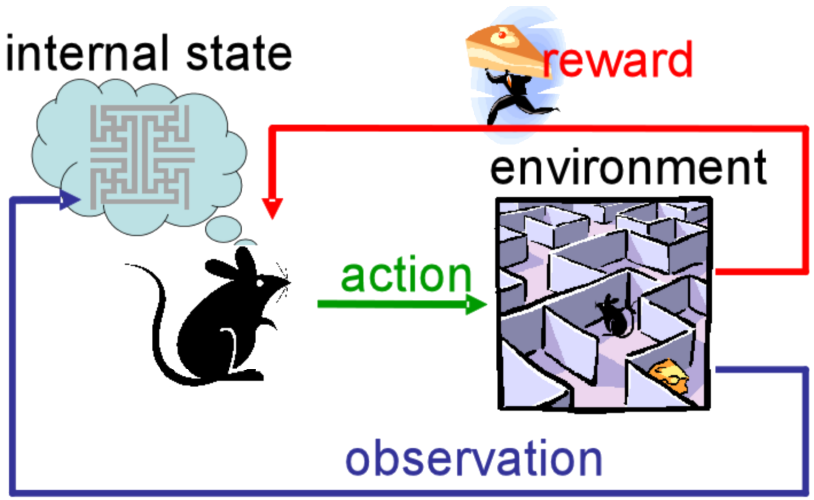
- (artificial) agent can choose action, interacting with its environment.
- state s_t = (partial) informations about the environment at time t
- policy π = strategy = procedure followed to select actions
- objective = maximise the cumulated reward

Reinforcement Learning (RL)

Objective of Reinforcement learning

Automatic acquisition of competences for decision making in a complex and uncertain environment

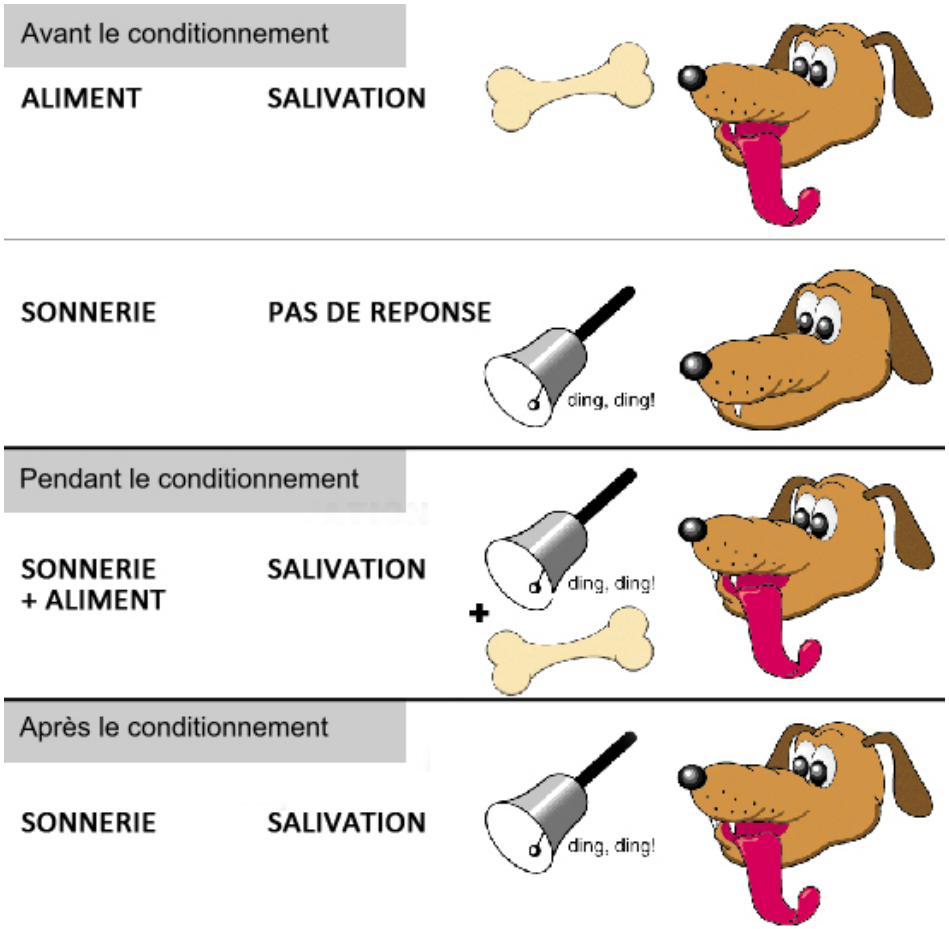
- birth around 1960's
- intersection of two fields
  - Computational Neurosciences
  - Experimental Psychology : adaptative behaviour



Experimental Psychology

Animal conditioning (Pavlov, 1904)

- Reinforcement Learning linked to the way animals learn
- The term "reinforcement" was first used by Pavlov in 1927



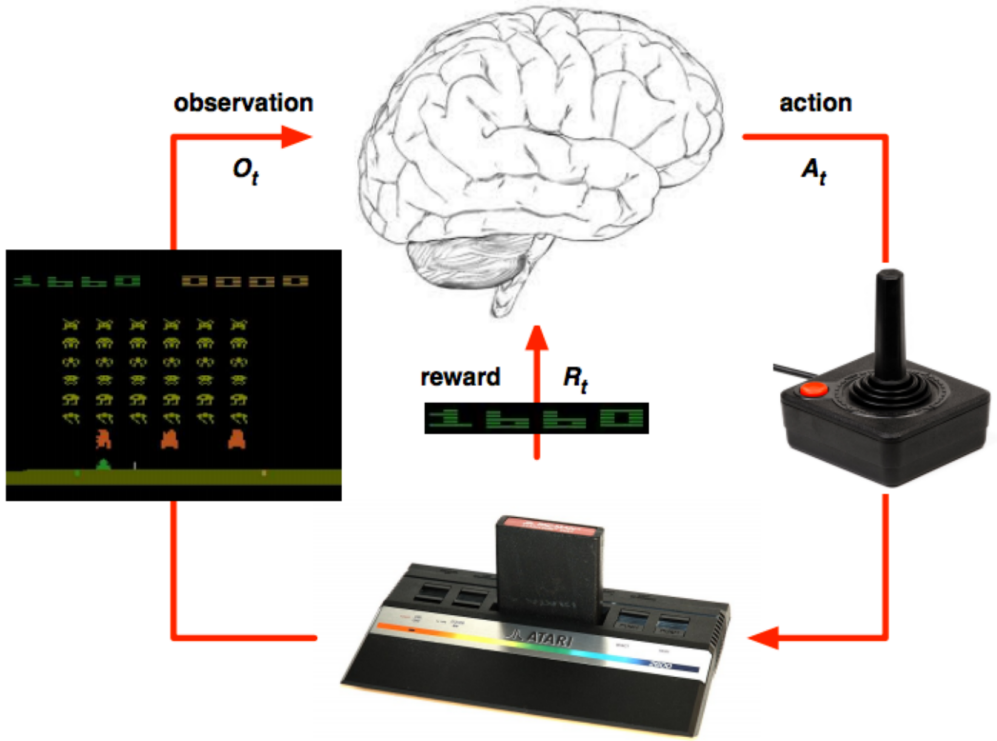
Experimental Psychology

From the dog's perspective. . .

- Reinforcement Learning linked to the way humans learn



Application : Games



Source : David Silver

- Each action influences the agent's future state
- Success is measured by a reward signal (e.g., score)
- Goal is to select actions to maximize future reward

Application : Games

- Atari Breakout game.
- actions : left / right
- (occasional) feedback information = score

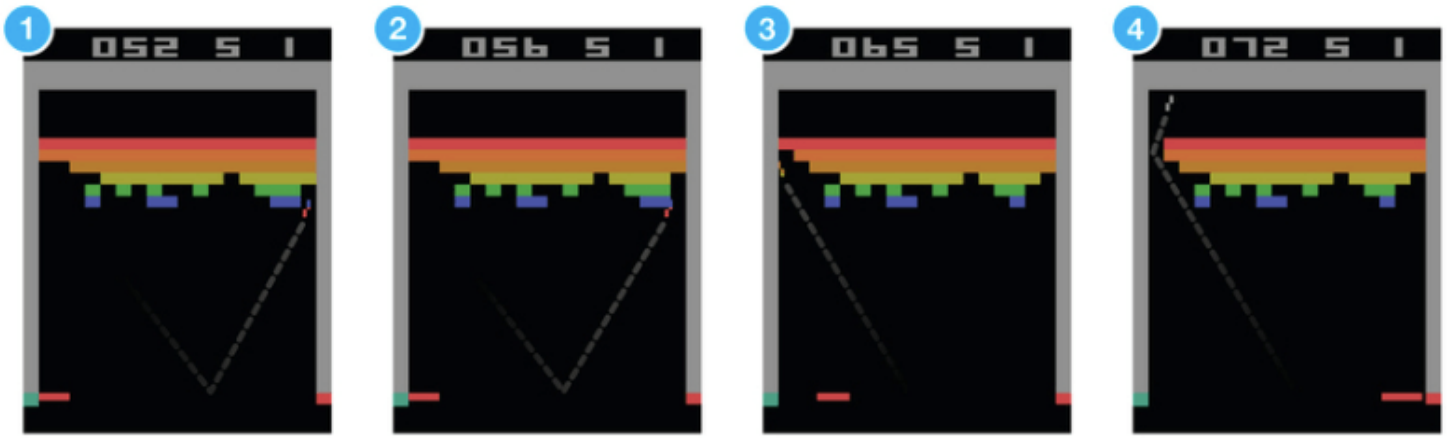


Image credit: DeepMind.

- history : Checkers (Samuel, 1952), TD-Gammon (Tesauro, 1992), . . .

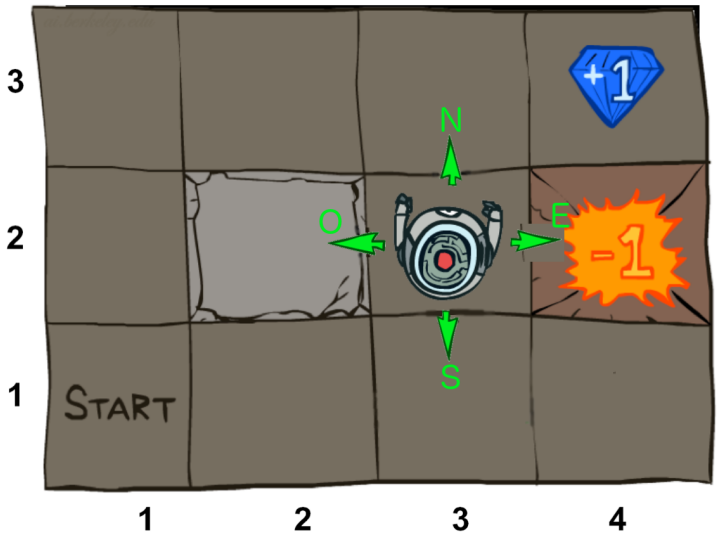
Application : Robots



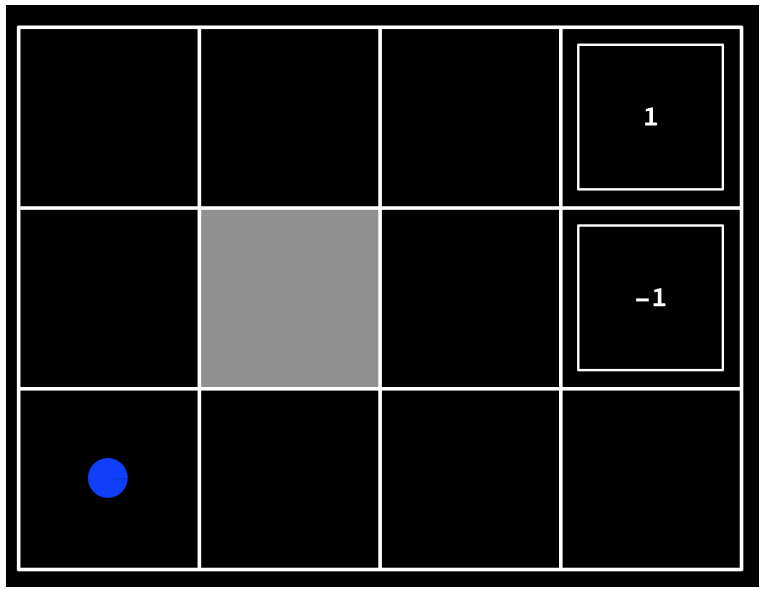
- observations = positions and angles of moving parts
- actions = forces, couples, ...
- reinforcement = keep balance / move / specific task / ...
- requires computer simulations for the learning phase
- `https://www.youtube.com/watch?v=VCdxqn0fcnE`

Toy Example (for practical sessions)

- `GridWorld`



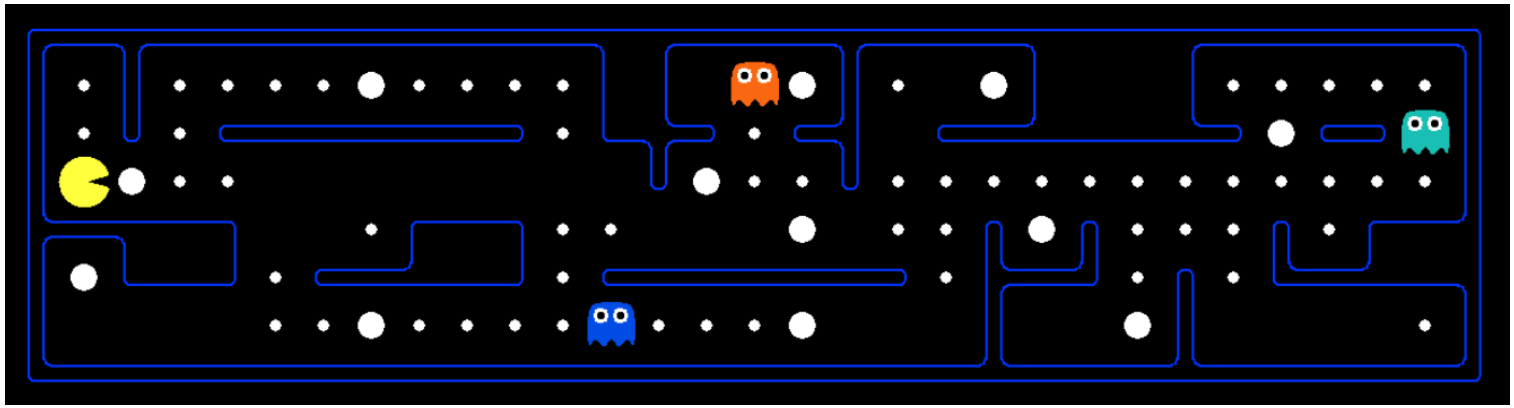
Source : `ai.berkeley.edu`



- we use this toy example to model
  - Markov Decision Processes (MDP)
  - Basic Reinforcement Learning (RL)
  - Q-Learning

Toy Example (for practical sessions)

- `PacMan`



Source : `ai.berkeley.edu`

- we use this toy example to model
  - Q-Learning
  - approximate Q-Learning
  - deep Q-Learning

Menu

1 Introduction

2 Markov Decision Processes
- Definitions
- Policies
- Rewards
- (q-)Value Function

3 Bellman Equations

4 Value Iteration

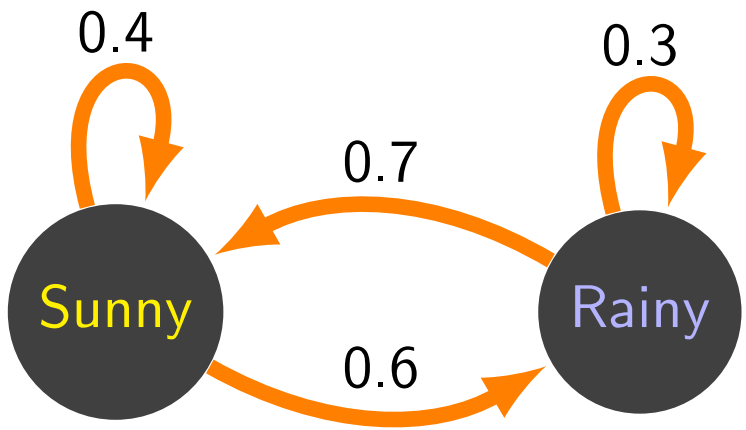
5 Policy Iteration

Markov chain

Definition : Markov chain

A Markov chain is an oriented graph whose nodes are states ($\in \mathcal{S}$) and edges are weighted by the transition probabilitites ($\in \mathcal{T}$) from one state to another.

- example : weather forecast



- no actions nor rewards, only transition probabilities between states

Markov chain and Markov property

- Can be used to model a discrete-time dynamical system

$$(s_t)_{t\in\mathbb{N}},$$

  where $s_t \in \mathcal{S}$ = state space.
- S_t is a random variable
- s_t is a realization of S_t

Definition : Markov (memoryless) property

The transition probabilities $T \in \mathcal{T}$ for a Markov Chain,

$$T(s, s') = \mathbb{P}(S_{t+1} = s' | S_t = s),$$

only depend on S_t, not on the past history $(S_{t-1}, S_{t-2}, \ldots)$.

Markov chain and Markov property

Markov property rephrased

- Given the present state, the future and the past are independent



Andrey Markov
(1856-1922)

Markov Decision Process (MDP)

Definition : Markov Decision Process (MDP)

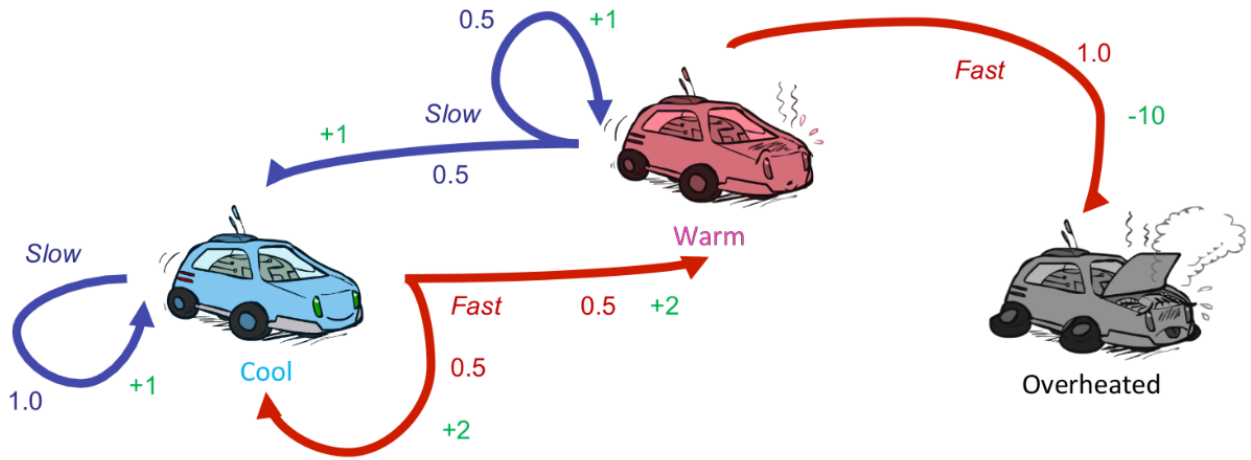
A Markov Decision Process (MDP) is a discrete time stochastic control process characterized by a 5-tuple $\{\mathcal{S}, \mathcal{A}, \mathcal{T}, T, R\}$.

- $\mathcal{S}$ = state space
- $\mathcal{A}$ = actions space
- $\mathcal{T}$ = transitions space
- T = transition function (conditional probability)
- R = reward function

Markov Decision Process (MDP)

Illustrative example

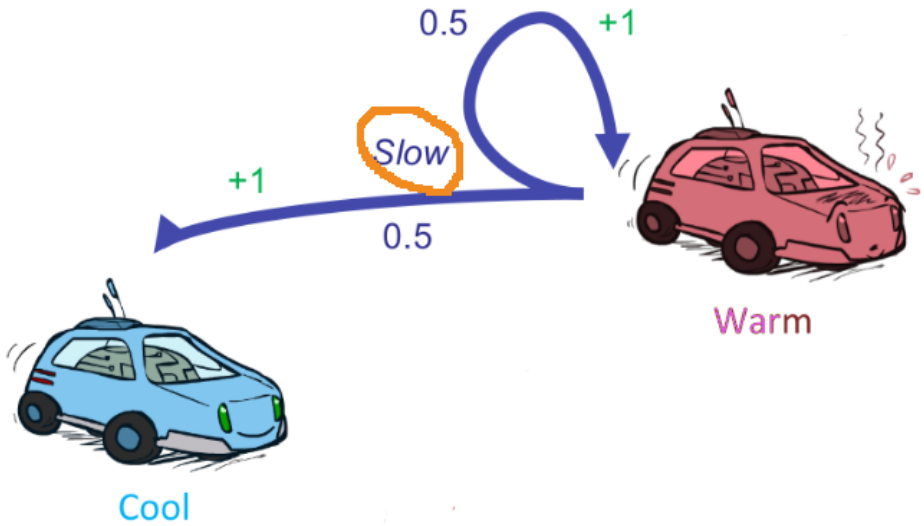
- we assume that the system is fully observable
- a transition graph summarizes the dynamics of a (finite) MDP



- $\mathcal{S}$ = state space= {Cool, Warm, Overheated}
- $\mathcal{A}$ = actions space = {Slow, Fast}
- $\mathcal{T}$ = transitions space = {(Warm, Slow, Warm), . . . }

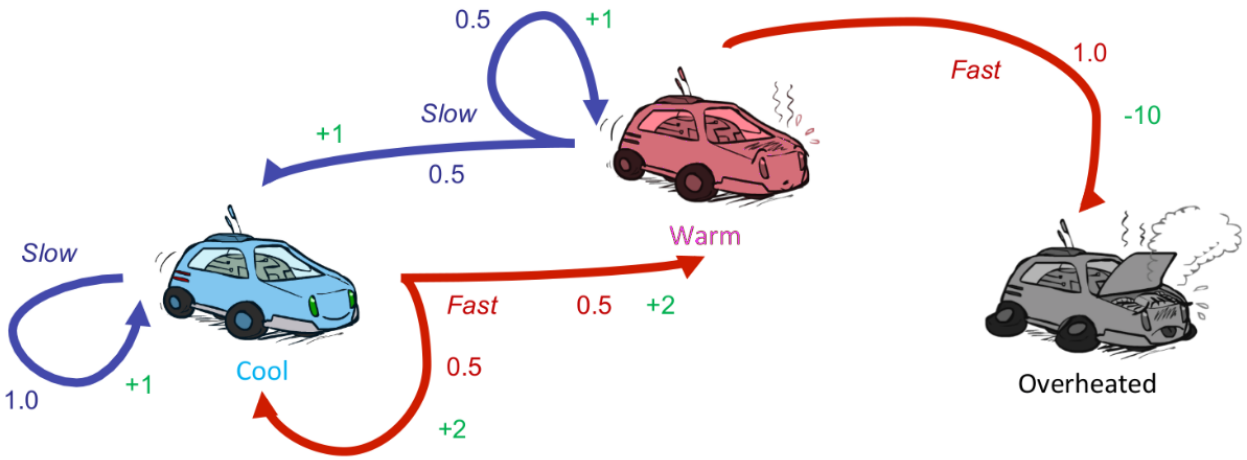
Markov Decision Process (MDP)

Transitions are stochastic !



- closer look at action Slow from state Warm
- does not always lead to the same state !
  - $\mathbb{P}(S_{t+1} = \text{Warm} | S_t = \text{Warm}, A_t = \text{Slow}) = 0.5$
  - $\mathbb{P}(S_{t+1} = \text{Cool} | S_t = \text{Warm}, A_t = \text{Slow}) = 0.5$

Markov Decision Process (MDP)



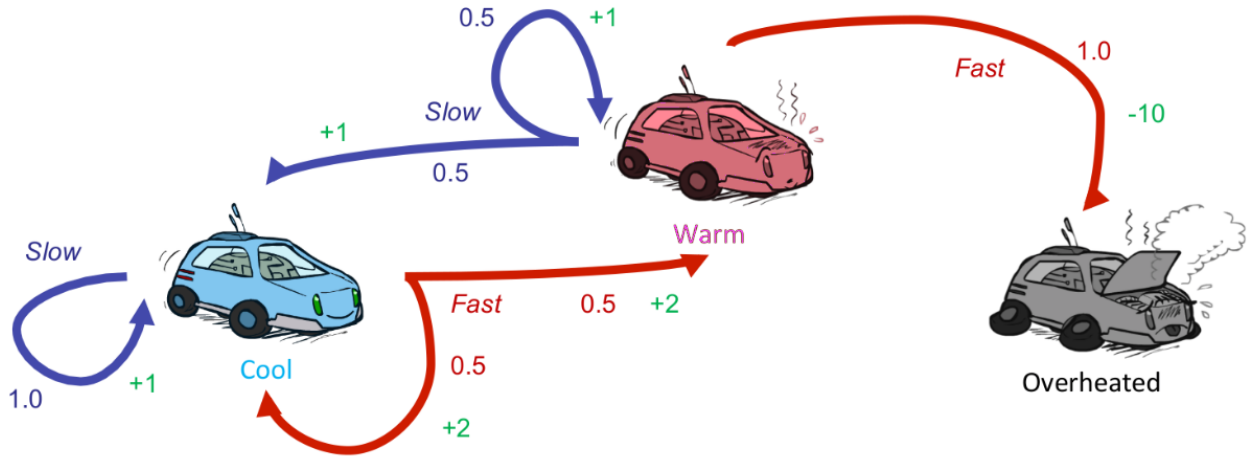
- T : transition function (conditional probability)

$$T : \mathcal{T} \rightarrow [0,1] : \{s, a, s'\} \rightarrow T(s, a, s') = \mathbb{P}(S_{t+1} = s' | A_t = a, S_t = s)$$

- R : reward function

$$R : \mathcal{T} \rightarrow \mathbb{R} : \{s, a, s'\} \rightarrow R(s, a, s')$$

Markov Decision Process (MDP)



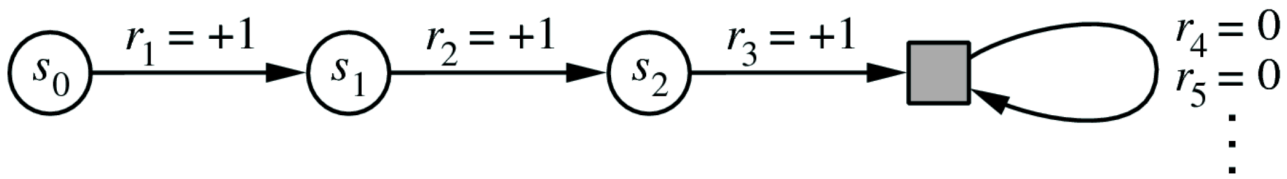
- T are known, for example :
  - $\mathbb{P}(\mathit{Warm}|\mathit{Slow}, \mathit{Warm}) = 0.5$
  - $\mathbb{P}(\mathit{Cool}|\mathit{Slow}, \mathit{Warm}) = 0.5$
- R are known
  - $R(s', \mathit{Slow}, s) = (+1)$
  - $R(s', \mathit{Fast}, s) = \begin{cases} (-10) & \text{if } s' = \text{ overheated} \\ (+2) & \text{otherwise} \end{cases}$

Markov Decision Process (MDP)

Definition : absorbing state

A state $s \in \mathcal{S}$ of an MDP is an **absorbing state** if

$$T(s, a, s) = 1 \qquad \forall a \in \mathcal{A}$$



MDP and Markov property

- in general, the consequences of the actions could depend on all the past states, rewards and actions :

$$\mathbb{P}\{S_{t+1} = s; R_{t+1} = r | S_t = s_t, R_t = r_t, A_t = a_t, \ldots, S_0 = s_0, R_0 = r_0, A_0 = a_0\}.$$

Markov (memoryless) property for a Markov Decision Process

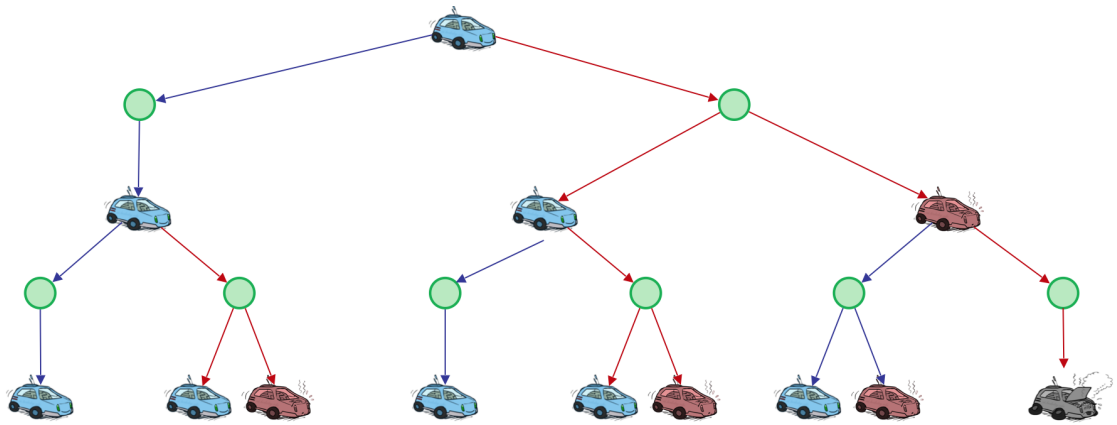
The consequences of an action only depend on the last step

$$\mathbb{P}\{S_{t+1} = s; R_{t+1} = r | S_t = s_t, R_t = r_t, A_t = a_t\}.$$

Thus the agent has no interest in looking at the full history.

MDP and trees

- can also be represented as a tree



- In a finite MDP, $\mathcal{S}$, $\mathcal{A}$ and $\mathcal{T}$ all have a finite number of elements.
- abstract and flexible framework

MDP and Policies

Definition : decision rule at time t

A decision rule is a function that determines the action law in any state at time t ($t \in \mathbb{N}$)

- deterministic : gives the action a to choose in state s

$$\pi_t : \mathcal{S} \to \mathcal{A} : s \mapsto a = \pi_t(s)$$

- **stochastic** : gives the probability to choose action a in s

$$\pi_t : \mathcal{S} \times \mathcal{A} \to \mathbb{R} : (s, a) \mapsto \pi_t(s, a)$$

MDP and Policies

Definition : policy

A policy (or strategy) is a sequence of decision rules

$$\pi = (\pi_0, \pi_1, \pi_2 \ldots).$$

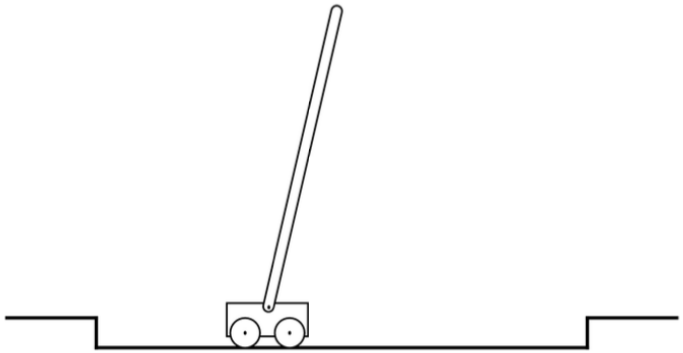
- policy $\pi \in \Pi$ thus defines the behaviour of an agent
  - deterministic $\pi(s)$: defines action a to perform in state s (with $\mathbb{P} = 1$)
  - **stochastic** $\pi(s, a)$: gives the probability to select a from state s, thus defining a distribution over actions given states,

$$\pi(a, s) = \pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$$

- any policy can be stationary ($\pi_t = \pi$, $\forall t$) or not

MDP - Episodic and continuing tasks

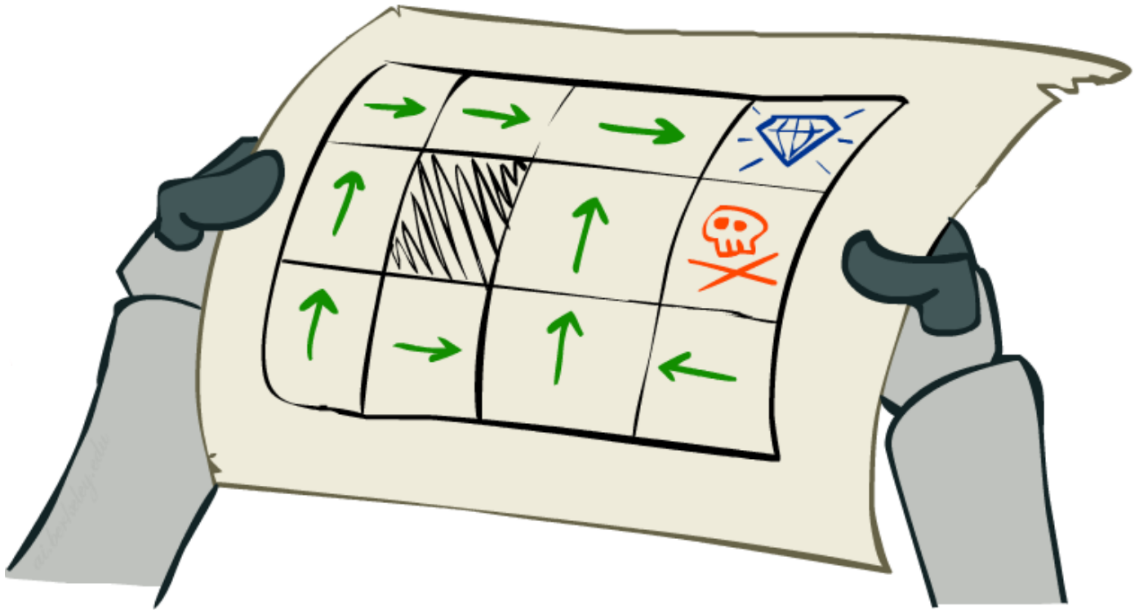
- episodic setting :
  - in every episode, the agent acts for some finite number of steps, starting from a fixed starting state s_0 until a terminal state
  - the state can be reset to s_0 after each episode
- continuing setting
  - no terminal state
  - "never ends"...



`gymnasium.farama.org/environments/classic_control/cart_pole`

How to "solve" an MDP ?

- goal : find the optimal policy π^*



Source : ai.berkeley.edu

- to find the optimal policy π^*, we have to maximize some reward

MDP and rewards

Reward hypothesis

All the purposes can be represented by the expected value of cumulated sum of a received scalar signal (return) which the agent tries to maximize.

- expected cumulated reward from current state $S_t = s$
  - after N steps (finite horizon)

$$\mathbb{E}\left[R_{t+1} + R_{t+2} + \ldots + R_{t+N} | S_t = s\right]$$

  - after infinite number of steps (infinite horizon)

$$\mathbb{E}\left[\sum_{i=0}^{\infty} R_{t+i+1} | S_t = s\right]$$

- convergence issues, need another definition

MDP and rewards

Definition : Return

The return is the expected deprecated reward from current state $S_t = s$

$$\mathbb{E}[\underbrace{\sum_{i=0}^{\infty} \gamma^i R_{t+i+1}}_{:=G_t} | S_t = s]$$

where $\gamma \in [0, 1[$ is the discount factor

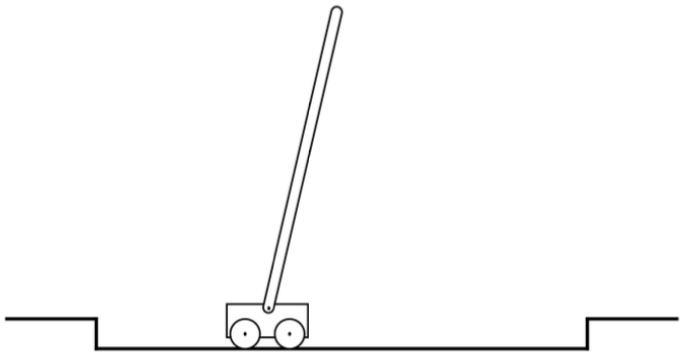
- γ can be seen as a trick to allow convergence of this series

$$\mathbb{E}[\sum_{i=0}^{\infty} \gamma^i R_{t+i+1} | S_t = s] \leq \frac{R_{\max}}{1-\gamma}$$

where $R_{\max}$ is the maximum reward over all states and time steps.

- $\gamma \to 0$: immediate reward matters ("myopic" agent)
- $\gamma \to 1$: all rewards matter equally ("far-sighted" agent)

MDP and rewards : example



- various possibilities to define rewards
  - episodic : $R_t = 0$ if falls, 1 otherwise,
    rewards keep growing as long as pole does not fall
  - continuing : $R_t = -1$ when falls, 0 otherwise,
    rewards set to 0 after the pole has fallen
- cumulated rewards
  - episodic : higher if efficient policy, possibly $\rightarrow \infty$
  - continuing : same for any policy
- deprecated rewards with ($\gamma \in]0, 1]$)
  - episodic and continuing : higher (bounded) reward for efficient policy

Markov Decision Process (MDP)

Definition : Value Function for policy π

For a given policy π, a value function

$$v^\pi : \mathcal{S} \to \mathbb{R}$$

associates to a state $s \in \mathcal{S}$ the value of the expected (deprecated) reward obtained by following π from s.

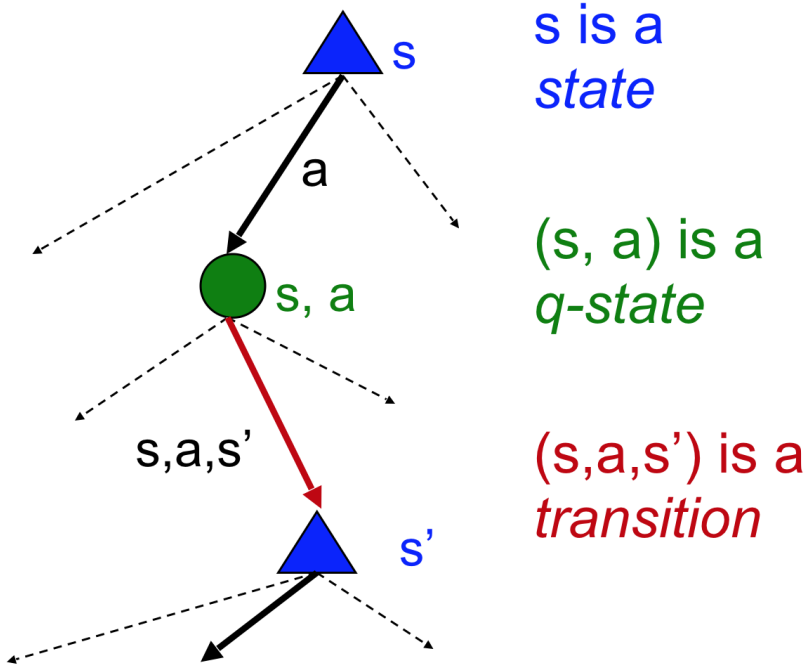
$$v^\pi(s) := \mathbb{E}^\pi[G_t | S_t = s] = \mathbb{E}^\pi[\sum_{i=0}^{\infty} \gamma^i R_{t+i+1} | S_t = s] \qquad \forall s \in \mathcal{S}$$

- the expectation $\mathbb{E}^\pi$ is conditional to the stochastic trajectories generated by the policy π

Markov Decision Process (MDP)

States and q-states

- associate a value $q(s, a)$ to each action in each state
- in q-state : action a has been chosen but not yet executed



Markov Decision Process (MDP)

Definition : q-Value Function for policy π

For a given policy π, the value function

$$q^{\pi}(s, a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$$

is defined by

$$q^{\pi}(s, a) = \mathbb{E}^{\pi}[G_t | S_t = s, A_t = a]$$

Markov Decision Process (MDP)

- values vs. q-values
  - $v^\pi(s) =$ value of a state s for a given policy π

$$v^\pi(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

  - $q^\pi(s, a) = q$-value of a pair (s, a) for a given policy π

$$q^\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a, \pi]$$

- policy evaluation vs. policy optimisation
  - policy evaluation : evaluate a policy π

$$\rightarrow \text{calculate} \quad v^\pi(s) \quad \forall s$$

  - policy optimisation : find the best policy π^*

$$\rightarrow \text{find} \quad q^*(s, a)$$

Menu

1 Introduction

2 Markov Decision Processes

3 Bellman Equations
- Recursive version
- Optimal version

4 Value Iteration

5 Policy Iteration

Markov Decision Process (MDP)

Value Function and recurrence relationship

- from the definition

$$G_t := \sum_{i=0}^{\infty} \gamma^i R_{t+i+1}$$

we can deduce the recurrence

$$G_t = R_{t+1} + \gamma G_{t+1}$$

- the value function for policy π can thus be decomposed into 2 parts:

$$\begin{aligned} v^\pi(s) &= \mathbb{E}^\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}^\pi[R_{t+1} + \gamma G(S_{t+1}) | S_t = s] \\ &= \mathbb{E}^\pi[R_{t+1} | S_t = s] + \gamma \mathbb{E}^\pi[G(S_{t+1}) | S_t = s] \end{aligned}$$

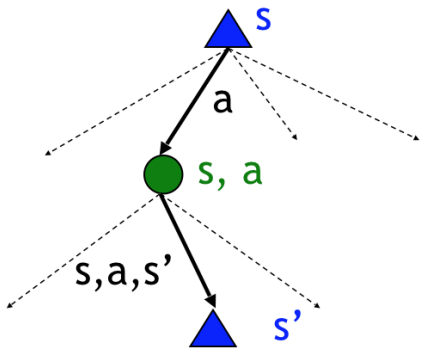
Markov Decision Process (MDP)

Resursive form of Bellman's equation (1957)

Bellman Equations (recursive form)

To evaluate a given policy π,

$$v^{\pi}(s) = \sum_{a} \pi(a,s) \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v^{\pi}(s')] \quad \forall s \in \mathcal{S}.$$



- immediate reward for transition : $R(s,a,s')$
- discounted value of successor : $\gamma v^{\pi}(s')$

The value of the successors s' are "retropropagated" to s

Markov Decision Process (MDP)

Optimal Policies and Optimal Value Functions

- use the value functions to define a partial ordering over policies, and to define the optimal policy

Definition : Partial ordering of policies

Assuming policies are comparable, a policy π dominates π' ($\pi \succ \pi'$) if

1. $\forall s,\ v^{\pi}(s) \geq v^{\pi'}(s)$
2. $\exists s :\ v^{\pi}(s) > v^{\pi'}(s)$

Definition : Optimal policy

A policy π^* is optimal if it is not dominated by any other:

$$\pi^*(s) = \arg\max_{\pi} v^{\pi}(s)$$

Markov Decision Process (MDP)

Optimal Policies and Optimal Value Functions

Definition : Optimal value function

$$\forall s \in \mathcal{S} : \qquad v^*(s) = v^{\pi^*}(s) = \max_{\pi} v^{\pi}(s)$$

Definition : Optimal q-value function

$$\forall s \in \mathcal{S}, a \in \mathcal{A} : \qquad q^*(s,a) = q^{\pi^*}(s,a) = \max_{\pi} q^{\pi}(s,a)$$

- there can be many optimal policies, but they all have the same v^*
- relationship between optimal value and optimal q-value :

$$v^*(s) = \max_{a} q^*(s,a)$$

Markov Decision Process (MDP)

- question : how to find v^*, π^*, q^*?
- answer : use Bellman Equations with optimisation

Bellman Equations (optimal version)

For a given state s, the optimal value function $v^*(s)$ is the solution of

$$v(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma v(s')]$$

and the corresponding optimal q-value function is

$$q^*(s, a) = \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma \max_{a'} q^*(s', a')]$$

Bellman's optimal equations

Link with operators and fixed-point theorems

- introducing the operator L

$$Lv^{\pi}(s) = \max_{a} \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v^{\pi}(s')]$$

- the optimal value $v^{*}(s)$ is the solution of

$$v^{\pi}(s) = Lv^{\pi}(s)$$

- the solution can be found using
  - exact methods (exact but slow)
  - approximate methods (faster)

Bellman's optimal equations

Computation of optimal value functions

- how to compute $v^*(s)$ in practice ?
- exact methods
  - solve explicitly this system of equations (for finite MDP) : slow !
- approximate methods
  - linear programming : slow !
  - dynamic programming = iterative methods using fixed-point theorems, which requires that L is a contraction operator
    - ★ value iteration
    - ★ policy iteration (= strategy iteration)

Menu

1 Introduction

2 Markov Decision Processes

3 Bellman Equations

4 Value Iteration

5 Policy Iteration

Markov Decision Process (MDP)

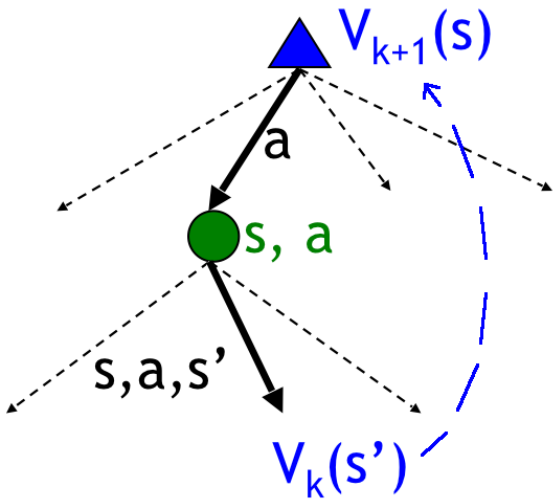
Value Iteration (VI)

- construct the sequence $\{v_0, v_1, \ldots, v_k, v_{k+1}, \ldots\}$ where

$$v_{k+1} = L v_k$$

- iterations to dynamically evaluate Bellman's equation

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma v_k(s')]$$



Markov Decision Process (MDP)

Value Iteration (VI) and fixed-point theorem

- fixed-point theorem applied to $v^*(s)$ also states that the fixed point can be obtained iteratively starting from any value $v_0(s)$

$$v^*(s) = \lim_{k \to \infty} v_k(s) \qquad \forall s \in \mathcal{S}$$

i.e., when convergence is reached, the Bellman equation will hold for every state: $v_k(s) = v_{k+1}(s) = v^*(s)$.

Markov Decision Process (MDP)

Value Iteration (VI) and stopping criterion

- consequently, if we define as stopping criterion

$$\|v_{k+1}(s) - v_k(s)\|_\infty < \epsilon \frac{1-\gamma}{\gamma}$$

- then we have a bound on the error at the next step

$$\|v_{k+1}(s) - v^*(s)\|_\infty < \epsilon$$

Markov Decision Process (MDP)

Value Iteration (VI)

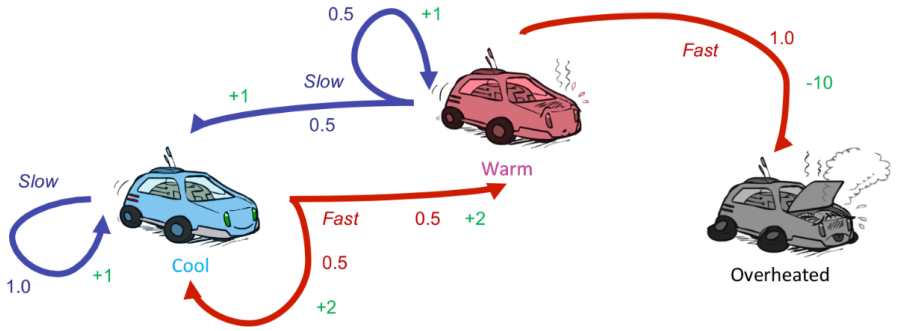
Algorithm 1 Value Iteration

Initialize $v_0(s)$ for all states, T and R are known
$k = -1$
repeat
  $k = k + 1$
  for $s \in \mathcal{S}$ **do**
    $v_{k+1}(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma v_k(s')]$
  end for
until $\|v_{k+1}(s) - v_k(s)\| < \epsilon$
return v_k

- all actions a from all states s to all states s' must be tried : $O(S^2A)$

Markov Decision Process (MDP)

Value Iteration (VI) - numerical example



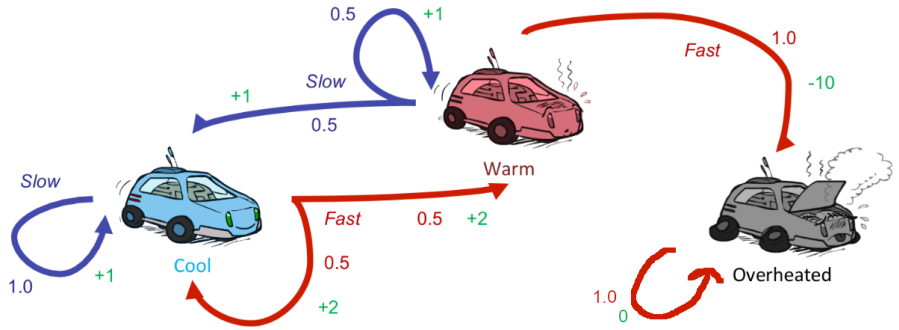
- starting point : all values $v_0(s)$ initialized to 0

	C	W	O
v_0	0	0	0

- we take $\gamma = 0.5$

Markov Decision Process (MDP)

Value Iteration (VI) - numerical example

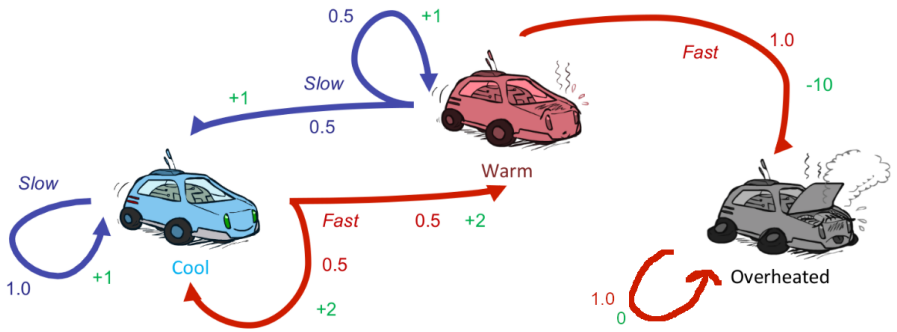


- first round of updates : v_1 from v_0

$$
\begin{aligned}
v_1(\mathrm{c}) &= \max\{1(1+0.5\times 0), 0.5(2+0.5\times 0)+0.5(2+0.5\times 0)\} \\
&= \max\{1,2\} = 2 \\
v_1(\mathrm{w}) &= \max\{0.5(1+0.5\times 0)+0.5(1+0.5\times 0), 1(-10+0.5\times 0)\} \\
&= \max\{1,-10\} = 1 \\
v_1(\mathrm{o}) &= \max\{0\} = 0
\end{aligned}
$$

Markov Decision Process (MDP)

Value Iteration (VI) - numerical example

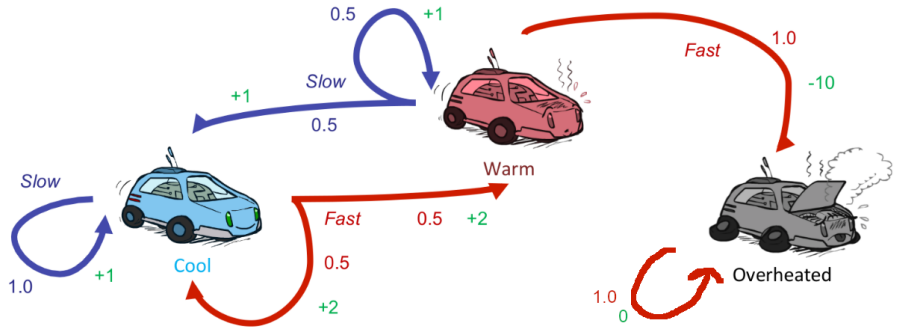


- first round of updates : v_1 from v_0

	C	W	O
v_0	0	0	0
v_1	2	1	0

Markov Decision Process (MDP)

Value Iteration (VI) - numerical example

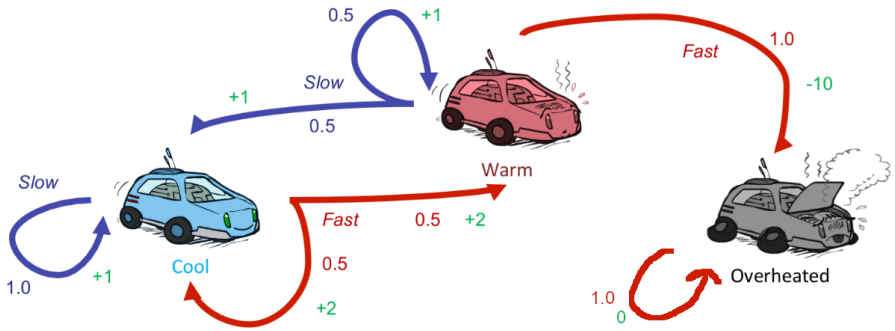


- second round of updates : v_2 from v_1

$$
\begin{aligned}
v_2(\mathrm{c}) &= \max\{1(1+0.5\times 2), 0.5(2+0.5\times 2)+0.5(2+0.5\times 1)\} \\
&= \max\{2, 2.75\} = 2.75 \\
v_2(\mathrm{w}) &= \max\{0.5(1+0.5\times 2)+0.5(1+0.5\times 1), 1(-10+0.5\times 0)\} \\
&= \max\{1.75, -10\} = 1.75 \\
v_2(\mathrm{o}) &= \max\{0\} = 0
\end{aligned}
$$

Markov Decision Process (MDP)

Value Iteration (VI) - numerical example



- second round of updates : v_2 from v_1

	C	W	O
v_0	0	0	0
v_1	2	1	0
v_2	2.75	1.75	0

Menu

1 Introduction

2 Markov Decision Processes

3 Bellman Equations

4 Value Iteration

5 Policy Iteration

Markov Decision Process (MDP)

Policy Iteration (PI) = Strategy Iteration (SI)

- if we go back to Bellman's optimal equation

$$v^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v^*(s')]$$

- this also defines the optimal policy

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v^*(s')]$$

  which is a fixed point equation on the space of stationary policies.

- it is not as straightforward to solve this iteratively

$$\pi_0 \to \pi_1 \to \ldots \to \pi_k \to \ldots$$

  because we need to get a value function v^{π_k} from the policy π_k

Markov Decision Process (MDP)

Policy Iteration (PI) = Strategy Iteration (SI)

- two steps, after initialization of $v_0^\pi(s)$
  1. policy evaluation : solve (no max)

$$v_k^{\pi_k} = L_{\pi_k} v_k^{\pi_k}$$

equivalent to iterate

$$\boxed{v_{k+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s')[R(s, \pi_k(s), s') + \gamma v_k^{\pi_k}(s')]}$$

  2. policy improvement (in a greedy way)

$$\pi_{k+1} = \arg\max_{\pi \in \Pi} \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma v_k^{\pi_k}(s')] \quad \forall s \in \mathcal{S}$$

- stop when policy does not change anymore $(\pi_k = \pi_{k+1})$

Markov Decision Process (MDP)

Policy Iteration (PI) = Strategy Iteration (SI)

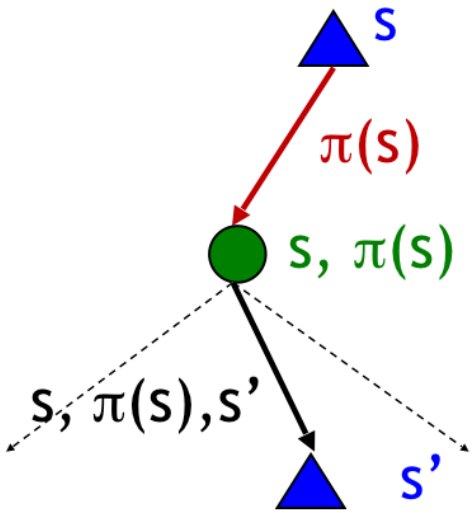
Algorithm 2 Policy Iteration

Initialize π_0 with some policy
$k = -1$
repeat
  $k = k + 1$
  for $s \in \mathcal{S}$ **do**
    solve $v^{\pi_k}(s) = \sum_{s'} T(s, \pi_k(s), s')[R(s, \pi_k(s), s') + \gamma v^{\pi_k}(s')]$
  end for
  for $s \in \mathcal{S}$ **do**
    $\pi_{k+1} = \underset{\pi \in \Pi}{\arg\max} \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma v^{\pi_k}(s')]$
  end for
until $\pi_{k+1}(s) = \pi_k(s)$
return v_k, π_k

Markov Decision Process (MDP)

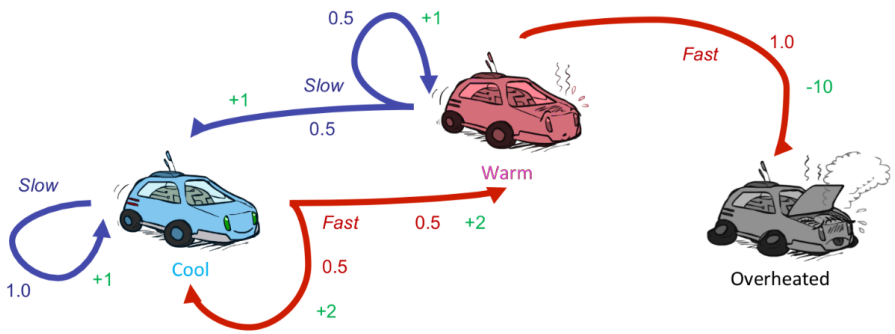
Policy Iteration (PI)

- the policy $\pi(s)$ is often more important than the precise value $v^*(s)$
- policy iteration can converge even though the values did not
- however it requires policy evaluation at each round, which can take a long time



Markov Decision Process (MDP)

Policy Iteration (PI) - numerical example

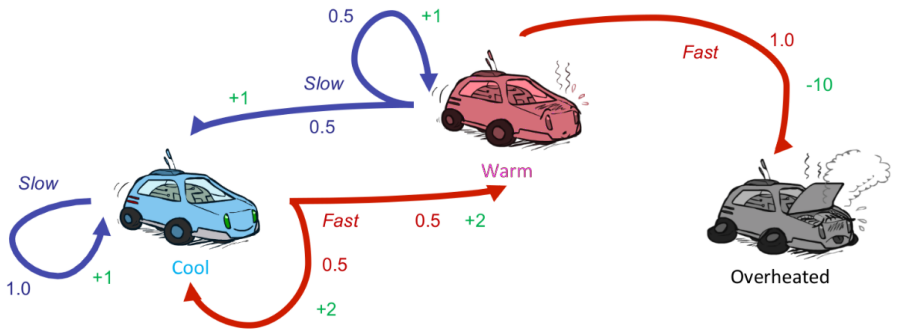


- starting point :
  - initial policy $\pi_0 =$ "always go slow"
  - all values $v^{\pi_0}(s)$ initialized to 0
- assume $\gamma = 0.5$

	c	w	o
π_0	slow	slow	-

Markov Decision Process (MDP)

Policy Iteration (PI) - numerical example



- first round of updates : (1) policy evaluation

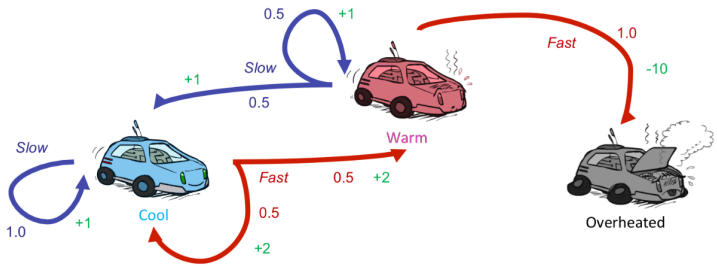
$$\begin{aligned} v^{\pi_0}(\mathrm{c}) &= 1(1 + 0.5 \times v^{\pi_0}(\mathrm{c})) \\ v^{\pi_0}(\mathrm{w}) &= 0.5(1 + 0.5 \times v^{\pi_0}(\mathrm{c})) + 0.5(1 + 0.5 \times v^{\pi_0}(\mathrm{w})) \end{aligned}$$

- solving these equations yields

$$v^{\pi_0}(\mathrm{c}) = 2 \qquad v^{\pi_0}(\mathrm{w}) = 2$$

Markov Decision Process (MDP)

Policy Iteration (PI) - numerical example

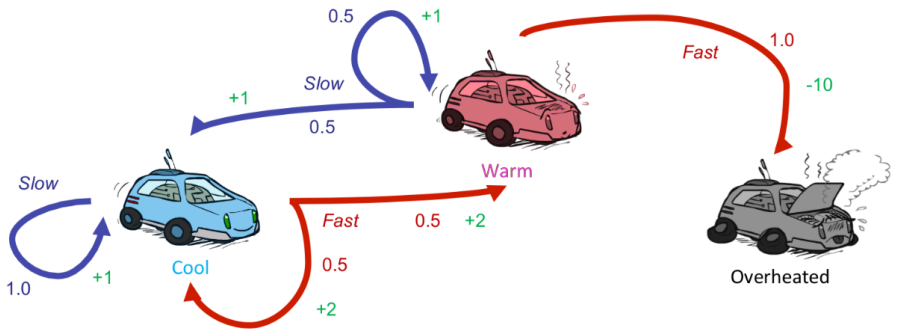


- first round of updates : (2) policy improvement

$$\begin{aligned}
\pi_1(\mathrm{c}) &= \arg\max\{\mathrm{slow} : 1(1+0.5\times 2), \\
&\qquad \mathrm{fast} : 0.5(2+0.5\times 2)+0.5(2+0.5\times 2)\} \\
&= \arg\max\{\mathrm{slow}:2, \mathrm{fast}:3\} = \mathrm{fast} \\
\pi_1(\mathrm{w}) &= \arg\max\{\mathrm{slow} : 0.5(1+0.5\times 2)+0.5(1+0.5\times 2), \\
&\qquad \mathrm{fast} : 0.5(-10+0.5\times 0)\} \\
&= \arg\max\{\mathrm{slow}:3, \mathrm{fast}:-10\} = \mathrm{slow}
\end{aligned}$$

Markov Decision Process (MDP)

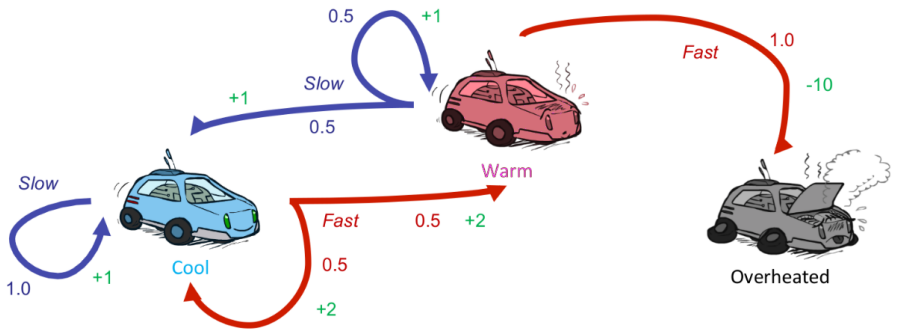
Policy Iteration (PI) - numerical example



	C	W	O
π_0	slow	slow	-
π_1	fast	slow	-

Markov Decision Process (MDP)

Policy Iteration (PI) - numerical example



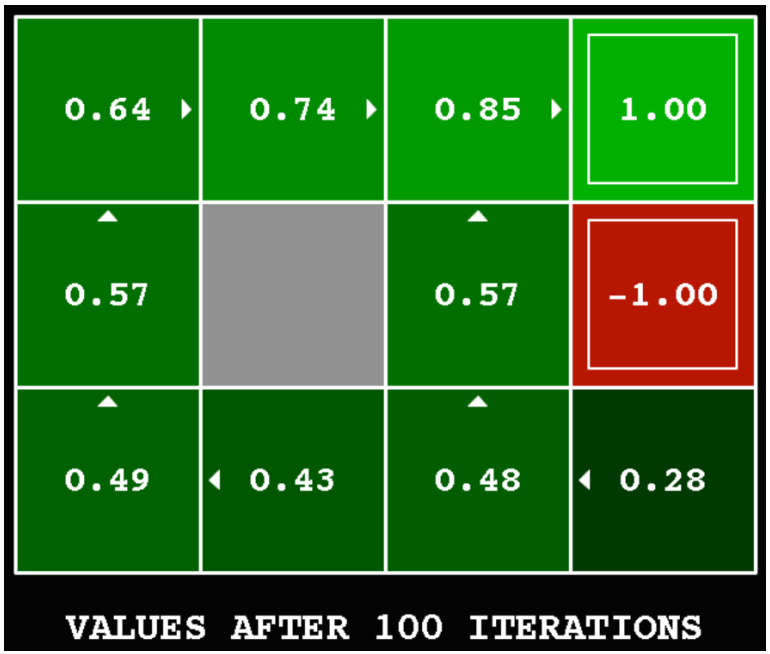
- second round of updates : policy evaluation and improvement yields

	C	W	O
π_0	slow	slow	-
π_1	fast	slow	-
π_2	fast	slow	-

- the policy converged already !

HANDS ON : MDP

- do **sections 1.1 → 1.4 of RL1**
- goal : explore MDP algorithms with simple labyrinths



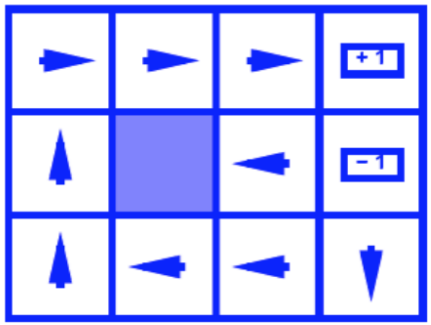
values $v^*(s)$



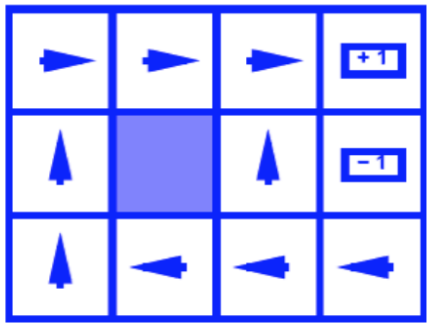
q-values $q^*(s, a)$

HANDS ON : MDP

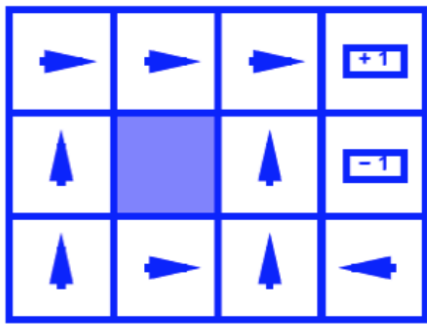
- Sometimes the agent takes too long to find the exit
- One can also add a living reward to speed up the process
- $\forall (s, a, s')$ with no reward, add a negative reward :
  - -0.01 (agent has plenty of time to get out)
  - -2.0 (hurry up !)



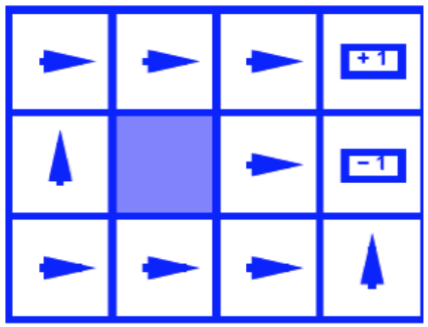
R(s) = -0.01



R(s) = -0.03



R(s) = -0.4



R(s) = -2.0

Source : ai.berkeley.edu